



Rotate Image by 90° Clockwise

Company: Microsoft

Round: Offline

Year: 2023

Difficulty: Medium

Topic: 2D Array, Matrix, In-Place Algorithm

Source: <https://www.designa.in/8942/tcs-coding-questions-answers-october-2022-minimum-operation>

Problem Description

You are given an $N \times N$ matrix representing an image.

Your task is to **rotate the matrix by 90 degrees clockwise**.

The rotation must be done **in-place**, which means:

- Modify the original matrix directly.
- Do **not** create another 2D matrix.
- Use only a small amount of extra memory.

Input Format

- The first line contains an integer N , representing the size of the matrix.
- The next N lines contain N integers representing the matrix.

Output Format

Print the matrix after rotating it 90 degrees clockwise.

Sample Input

```
3
1 2 3
4 5 6
7 8 9
```

Sample Output

```
7 4 1
8 5 2
9 6 3
```

Explanation

Given the matrix:

```
1 2 3
4 5 6
7 8 9
```

We need to rotate it **90° clockwise**.

First, we **transpose** the matrix by swapping rows and columns:

1 4 7

2 5 8

3 6 9

Then, **reverse each row**:

7 4 1

8 5 2

9 6 3

So the final rotated matrix is:

7 4 1

8 5 2

9 6 3

The important point is that both operations are performed on the **same matrix**, so no extra 2D matrix is required.

Approach to Solve This Problem:

- Transpose the matrix by swapping `matrix[i][j]` with `matrix[j][i]`.
- After transposing, reverse every row of the matrix.
- The resulting matrix will be the original matrix rotated 90° clockwise.
- Both operations are performed in-place, so no extra 2D matrix is needed.

Java Solution:

```
1  import java.util.*;
2  public class Main {
3      public static void rotate(int[][] matrix) {
4          int n = matrix.length;
5          // Step 1: Transpose the matrix
6          for (int i = 0; i < n; i++) {
7              for (int j = i + 1; j < n; j++) {
8                  int temp = matrix[i][j];
9                  matrix[i][j] = matrix[j][i];
10                 matrix[j][i] = temp;
11             }
12         }
13         for (int i = 0; i < n; i++) {
14             int left = 0;
15             int right = n - 1;
16             while (left < right) {
17                 int temp = matrix[i][left];
18                 matrix[i][left] = matrix[i][right];
19                 matrix[i][right] = temp;
20                 left++;
21                 right--;
22             }
23         }
24     }
25     public static void main(String[] args) {
26         Scanner sc = new Scanner(System.in);
27         int n = sc.nextInt();
```

```
28         int[][] matrix = new int[n][n];
29         for (int i = 0; i < n; i++) {
30             for (int j = 0; j < n; j++) {
31                 matrix[i][j] = sc.nextInt();
32             }
33         }
34         rotate(matrix);
35         for (int i = 0; i < n; i++) {
36             for (int j = 0; j < n; j++) {
37                 System.out.print(matrix[i][j] + " ");
38             }
39             System.out.println();
40         }
41         sc.close();
42     }
43 }
```

Solution:

The solution is to rotate the matrix using **two in-place operations**:

1. **Transpose the matrix:**

Swap the elements across the main diagonal. This changes rows into columns.

2. **Reverse each row:**

After transposing, reverse every row. This produces the required **90° clockwise rotation**.

[LinkedIn](#) | [YouTube](#) | [Instagram](#)

Complexity Analysis:

Time Complexity: $O(N^2)$

The matrix has $N \times N$ elements. We traverse the matrix during transposition and again while reversing each row.

Space Complexity: $O(1)$

The rotation is performed in-place by swapping elements, so no extra matrix or significant additional memory is used.